Optimal Relaxation Parameters for the Successive Over-Relaxation Method

Theory, Derivation, and Numerical Experiments

Biajid Chowdhury

November 8, 2026

Abstract

One of the most fundamental problems in computational science and engineering is solving a system of linear equations of the form

$$Ax = b.$$

For nearly two millennia, engineers, mathematicians and scientists have explored different approaches to solving such systems efficiently at an acceptable computational cost. From the elimination procedures of ancient mathematics to modern direct and iterative algorithms, this endeavor has produced numerous important developments and continues to motivate research in scientific computing.

Among these developments is the introduction of relaxation parameters into iterative methods and, more importantly, the problem of determining their optimal values. The successive over-relaxation (SOR) method provides a particularly interesting example, since a suitable choice of the relaxation parameter can substantially accelerate convergence, whereas a poor choice may lead to extremely slow convergence or even divergence.

The mathematical foundations of SOR emerged from several decades of research on iterative methods and numerical solutions of differential equations. An important breakthrough came through the work of David M. Young, whose doctoral research in 1950 and subsequent publications established a systematic theoretical framework for successive over-relaxation. For certain structured classes of matrices, this theory provides an explicit expression for the optimal relaxation parameter in terms of the spectral radius of the Jacobi iteration matrix. For general matrices, however, determining the optimal parameter remains a more difficult problem, and no comparably simple universal formula is available.

In this note, we study the mathematical formulation of the Jacobi, Gauss–Seidel, and successive over-relaxation methods, with particular attention to their convergence properties. We derive the optimal SOR parameter for an important class of tridiagonal matrices arising from finite difference discretizations of boundary value problems. Finally, we implement the methods in MATLAB and compare their numerical performance, examining the relationship between relaxation parameters, spectral radii, convergence rates, and computational accuracy.

The development of optimal relaxation methods represents a remarkable chapter in the history of numerical computation. Its mathematical elegance, practical significance, and long historical development deserve closer attention. It is therefore somewhat surprising that the derivation of optimal relaxation parameters receives relatively limited treatment in many undergraduate textbooks, and instructors often overlook it. We hope that the present discussion will help illuminate both the mathematics and the significance of this important subject.

Contents

1 Introduction **3**

2 Iterative Procedures for Solving Systems of Linear Equations **5**

2.1 A Simple Example 6

2.2 The Jacobi Iterative Method 7

2.3 The Gauss–Seidel Iterative Method 8

2.4 A Common Mathematical Structure 10

3 The Successive Over-Relaxation Method **12**

3.1 Introducing the Relaxation Parameter 13

3.2 Successive Over-Relaxation as a Modification of Gauss–Seidel 14

3.3 A Simple Numerical Example 15

3.4 Matrix Formulation of SOR 15

4 Convergence Analysis and Optimal Relaxation Parameters **17**

4.1 A Necessary Condition for SOR Convergence 18

4.2 An Immediate Consequence for Optimal Relaxation 19

4.3 Convergence of SOR for Symmetric Positive Definite Matrices 20

5 Derivation of the Optimal SOR Parameter **23**

5.1 The Relationship Between Jacobi and SOR Eigenvalues 24

5.2 Derivation of the Optimal Relaxation Parameter 27

5.3 Optimal SOR Parameter for a Finite-Difference Matrix 34

1 Introduction

From elementary school, we begin learning how to solve equations. In fact, the journey probably starts as early as the fourth grade, when we encounter a simple problem such as

$$4x = 8.$$

By middle school, we are already accustomed to word problems involving several unknown quantities. We translate the information given in words into a system of equations and then try to determine the unknowns. Without necessarily knowing its name, we are often using elementary forms of the elimination procedure that later becomes Gaussian elimination.

By high school, many students have encountered systems of linear equations, determinants, and perhaps even Cramer's rule. Thus, the mathematical problem we are going to investigate in this note is certainly nothing new to the reader. What changes is not the problem itself, but its size and the computational resources required to solve it.

Consider a system of n linear equations in n unknowns, written in matrix form as

$$Ax = b,$$

where $A \in \mathbb{R}^{n\times n}$, $x \in \mathbb{R}^n$ is the unknown vector, and $b \in \mathbb{R}^n$ is given.

For a system involving two or three unknowns, Gaussian elimination is straightforward. Even for a system of a few hundred equations, a modern computer can solve the problem with little difficulty. But what happens when the number of unknowns reaches several million? Such systems arise naturally in computational science and engineering, particularly when differential equations are approximated by finite difference or finite element methods.

The elimination procedure developed by Gauss and subsequently extended into the Gauss–Jordan method remains one of the fundamental achievements of numerical linear algebra. However, for a general dense $n \times n$ matrix, Gaussian elimination requires on the order of n^3 arithmetic operations and n^2 storage locations. As n becomes very large, these requirements can become prohibitively expensive. Although modern direct methods exploit sparsity and other special matrix structures to reduce these costs, it is natural to ask whether we can approach the problem differently.

What alternative might we consider?

Perhaps we could return to something familiar from our earlier mathematical education: factorization. Instead of solving $Ax = b$ directly, suppose we could write

$$A = LU,$$

where L is lower triangular and U is upper triangular. Then the original system becomes

$$LUx = b,$$

which can be solved through two triangular systems:

$$Ly = b, \qquad Ux = y.$$

This certainly simplifies the solution once the factors are known. But obtaining those factors is itself a computational problem. In fact, Gaussian elimination is precisely one of the principal procedures for constructing an LU factorization. Thus, factorization does not automatically remove the computational difficulty; rather, it provides another way of organizing the same fundamental calculation.

So we ask a different question:

Must we determine the solution in a finite sequence of elimination steps, or could we begin with an approximate solution and improve it repeatedly until it becomes sufficiently accurate?

This question leads us to iterative methods.

Instead of attempting to solve $Ax = b$ directly, an iterative method begins with an initial guess $x^{(0)}$ and constructs a sequence

$$x^{(0)},\ x^{(1)},\ x^{(2)},\ \dots$$

that, under suitable conditions, converges to the exact solution x^*:

$$\lim_{k\to\infty} x^{(k)} = x^*, \qquad Ax^* = b.$$

The idea is simple: start somewhere, make a correction, and repeat. The mathematical difficulty lies in designing a correction that guarantees convergence and, equally importantly, makes that convergence sufficiently fast.

Two classical examples are the Jacobi and Gauss–Seidel methods. Both construct successive approximations by rearranging the equations of the original system. Their simplicity makes them attractive, but their convergence can be painfully slow, particularly for large systems arising from the discretization of differential equations.

Can we accelerate the process?

One possibility is to modify the correction at each iteration by introducing a relaxation parameter ω. Rather than accepting the new approximation exactly as computed, we move toward it by an amount controlled by ω. This leads to the successive over-relaxation (SOR) method, a modification of Gauss–Seidel that can converge substantially faster when the parameter is chosen appropriately.

But this improvement introduces a new problem: *how should we choose ω?*

A parameter that works remarkably well for one matrix may perform poorly for another. Even for the same matrix, a small change in ω may significantly affect the number of iterations required for convergence. Therefore, the problem is not merely to introduce a relaxation parameter, but to understand whether an optimal value exists and how it can be determined mathematically.

This is the central question of the present note.

We begin by developing the Jacobi and Gauss–Seidel methods through the matrix splitting

$$A = D + L + U,$$

where D contains the diagonal entries of A, while L and U denote its strictly lower and strictly upper triangular parts, respectively. We then introduce the SOR method and derive its iteration matrix. Using the spectral radius, we investigate the convergence of these stationary iterative methods and examine how the relaxation parameter influences the convergence rate.

Our principal objective is to derive an explicit expression for the optimal SOR parameter for a particular class of structured matrices. We then apply the theory to a tridiagonal system obtained from the finite difference discretization of a second-order boundary value problem. In this setting, the eigenvalues of the Jacobi iteration matrix can be determined analytically, allowing us to obtain the optimal relaxation parameter in closed form.

Finally, we implement the Jacobi, Gauss–Seidel, and SOR methods in MATLAB and compare their numerical performance. Particular attention is given to iteration counts, convergence rates, and the distinction between iteration error and discretization error.

Our aim is not to present SOR merely as another algorithm for solving linear equations. Rather, we wish to understand the mathematical reasoning behind its acceleration, the role of eigenvalues in predicting convergence, and why the seemingly simple question of choosing a relaxation parameter leads to such an interesting problem in numerical linear algebra.

2 Iterative Procedures for Solving Systems of Linear Equations

Before introducing iterative methods for solving linear systems, let us begin with a rather elementary problem from computer science.

Suppose we are given the array

$$[1,\ 10,\ 2,\ 5,\ 9,\ -1]$$

and asked to determine whether the number 9 appears in it.

How should we proceed?

An enthusiastic reader familiar with algorithms might immediately suggest sorting the array and then performing a binary search. That is certainly a reasonable approach. But before choosing an algorithm, perhaps we should ask another question:

Are we going to perform this search only once, or will we need to search the same array repeatedly?

If the search is performed only once, sorting the entire array may be unnecessary. We can simply begin with the first element, compare it with 9, and continue until either the desired number is found or the end of the array is reached. This is the familiar linear search algorithm.

However, if we expect to search the same array repeatedly, perhaps every minute, the situation changes. In that case, it may be worthwhile to sort the array once and then use binary search for subsequent queries. By examining the middle element and repeatedly eliminating half of the remaining search interval, binary search requires only $O(\log n)$ comparisons, compared with $O(n)$ comparisons in the worst case for linear search.

The important lesson is that the choice of an algorithm depends on the circumstances in which it will be used. A method that is attractive for one computation may not be the most efficient choice for another.

A similar consideration arises when solving systems of linear equations. If the same coefficient matrix is used with many different right-hand sides, computing a matrix factorization once and reusing it may be particularly advantageous. On the other hand, for a very large sparse system that needs to be solved only once, constructing and storing such a factorization may be unnecessarily expensive.

What alternative do we have?

Instead of attempting to obtain the solution through a direct procedure, we might begin with an approximate solution and improve it repeatedly. If the approximations converge sufficiently close to the desired solution within our computational budget, we accept the result. Otherwise, we report that the prescribed convergence criterion has not been satisfied.

This is the fundamental idea behind an iterative method.

Of course, unlike searching an array, we are not examining a collection of candidate solutions one by one. Rather, we construct a sequence of approximations according to a mathematical rule designed to bring us closer to the solution.

Let us see how this works through a simple example.

2.1 A Simple Example

Consider the system

$$\begin{aligned} 4x + y &= 9, \\ x + 3y &= 7. \end{aligned} \tag{1}$$

In matrix notation,

$$\begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 9 \\ 7 \end{bmatrix}.$$

We could solve this system immediately using Gaussian elimination. In fact, its exact solution is

$$\boxed{x = \frac{20}{11}, \qquad y = \frac{19}{11}.}$$

But suppose we did not know the solution and wanted to construct it through successive approximations.

Rearranging the equations in (1), we obtain

$$\begin{aligned} x &= \frac{9 - y}{4}, \\ y &= \frac{7 - x}{3}. \end{aligned} \tag{2}$$

These equations suggest an iterative procedure.

Choose an initial approximation

$$x^{(0)} = 0, \qquad y^{(0)} = 0.$$

Now use the right-hand sides of (2) to construct new approximations.

For the first iteration,

$$\begin{aligned} x^{(1)} &= \frac{9 - y^{(0)}}{4} = \frac{9}{4} = 2.25, \\ y^{(1)} &= \frac{7 - x^{(0)}}{3} = \frac{7}{3} \approx 2.333333. \end{aligned}$$

For the second iteration,

$$\begin{aligned} x^{(2)} &= \frac{9 - y^{(1)}}{4} \approx 1.666667, \\ y^{(2)} &= \frac{7 - x^{(1)}}{3} \approx 1.583333. \end{aligned}$$

Continuing in this manner gives the following sequence of approximations:

Iteration k	$x^{(k)}$	$y^{(k)}$
0	0.000000	0.000000
1	2.250000	2.333333
2	1.666667	1.583333
3	1.854167	1.777778
4	1.805556	1.715278
5	1.821181	1.731481
Exact	1.818182	1.727273

Notice that the approximations move toward the exact solution. Although they do not approach it monotonically, the errors become progressively smaller.

The procedure we have just constructed is an example of the *Jacobi iterative method*.

But an important question remains:

Will this procedure always converge, regardless of the system of equations?

Unfortunately, the answer is no.

The convergence of an iterative method depends on the structure of the coefficient matrix and the particular iteration rule being used. We shall investigate these conditions mathematically after developing the general formulation.

2.2 The Jacobi Iterative Method

Consider the linear system

$$Ax = b, \tag{3}$$

where $A = (a_{ij}) \in \mathbb{R}^{n \times n}$.

Suppose that every diagonal entry of A is nonzero:

$$a_{ii} \neq 0, \qquad i = 1, \ldots, n.$$

The ith equation of the system is

$$a_{i1}x_1 + \cdots + a_{ii}x_i + \cdots + a_{in}x_n = b_i.$$

Solving for x_i gives

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^{n} a_{ij} x_j \right).$$

This suggests the iteration

$$\boxed{x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^{n} a_{ij} x_j^{(k)} \right).} \tag{4}$$

The essential feature of the Jacobi method is that every component of the new approximation is computed using only the components of the previous approximation.

Thus, during iteration $k+1$, the values

$$x_1^{(k+1)}, \ldots, x_n^{(k+1)}$$

are calculated independently from the vector $x^{(k)}$.

This makes the method conceptually simple and particularly suitable for parallel computation.

Matrix Formulation

Write

$$A = D + L + U,$$

where D is the diagonal part of A, L is its strictly lower triangular part, and U is its strictly upper triangular part.

Then

$$(D + L + U)x = b,$$

or equivalently,

$$Dx = b - (L + U)x.$$

Replacing the unknown vector on the right-hand side by the current approximation gives

$$Dx^{(k+1)} = b - (L + U)x^{(k)}.$$

Since D is invertible,

$$\boxed{x^{(k+1)} = -D^{-1}(L + U)x^{(k)} + D^{-1}b.} \tag{5}$$

Define

$$T_J = -D^{-1}(L + U), \qquad c_J = D^{-1}b.$$

The Jacobi iteration therefore takes the compact form

$$\boxed{x^{(k+1)} = T_J x^{(k)} + c_J.} \tag{6}$$

The matrix T_J is called the *Jacobi iteration matrix*.

As we shall see later, its eigenvalues determine whether the iteration converges.

2.3 The Gauss–Seidel Iterative Method

Let us return briefly to our elementary example.

Recall that the Jacobi method uses the previous approximation to calculate every component of the new vector.

But suppose we have already calculated a new value of x. Why should we continue using the old value when calculating y?

Would it not be more reasonable to use the most recently available information?

This simple observation leads to the Gauss–Seidel method.

For the system

$$\begin{aligned} 4x + y &= 9, \\ x + 3y &= 7, \end{aligned}$$

we now use

$$\begin{aligned} x^{(k+1)} &= \frac{9 - y^{(k)}}{4}, \\ y^{(k+1)} &= \frac{7 - x^{(k+1)}}{3}. \end{aligned} \tag{7}$$

Notice the difference: the second equation uses $x^{(k+1)}$, not $x^{(k)}$.

Starting again from

$$x^{(0)} = y^{(0)} = 0,$$

the first iteration gives

$$x^{(1)} = \frac{9}{4} = 2.25,$$
$$y^{(1)} = \frac{7 - 2.25}{3} \approx 1.583333.$$

The second iteration gives

$$x^{(2)} = \frac{9 - 1.583333}{4} \approx 1.854167,$$
$$y^{(2)} = \frac{7 - 1.854167}{3} \approx 1.715278.$$

Continuing the process, we obtain

Iteration k	$x^{(k)}$	$y^{(k)}$
0	0.000000	0.000000
1	2.250000	1.583333
2	1.854167	1.715278
3	1.821181	1.726273
4	1.818432	1.727189
5	1.818203	1.727266
Exact	1.818182	1.727273

Comparing these values with the Jacobi iterations, we observe that Gauss–Seidel converges considerably faster for this particular system.

This improvement comes from using the newly computed components immediately rather than waiting until the next iteration.

General Formulation

For a general system $Ax = b$, the Gauss–Seidel iteration is

$$\boxed{x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right).} \tag{8}$$

The first summation uses newly computed values, while the second uses values from the previous iteration.

Using the decomposition

$$A = D + L + U,$$

we can write

$$(D + L)x^{(k+1)} = b - Ux^{(k)}.$$

Therefore,

$$\boxed{x^{(k+1)} = -(D + L)^{-1} U x^{(k)} + (D + L)^{-1} b.} \tag{9}$$

Define

$$T_{GS} = -(D + L)^{-1} U, \qquad c_{GS} = (D + L)^{-1} b.$$

Then

$$\boxed{x^{(k+1)} = T_{GS}x^{(k)} + c_{GS}.} \tag{10}$$

Although the formula contains $(D + L)^{-1}$, we do not explicitly compute this inverse in practice. Instead, each iteration requires solving a lower triangular system, which can be accomplished efficiently through forward substitution.

2.4 A Common Mathematical Structure

The Jacobi and Gauss–Seidel methods may appear different in their componentwise formulations, but they share the same fundamental structure:

$$\boxed{x^{(k+1)} = Tx^{(k)} + c.}$$

Here, T is the iteration matrix and c is a fixed vector.

Suppose the iteration converges to a vector x^*. Taking limits gives

$$x^* = Tx^* + c.$$

Therefore,

$$(I - T)x^* = c.$$

For both Jacobi and Gauss–Seidel, this fixed-point equation is equivalent to the original system $Ax = b$.

But convergence is not automatic.

Let

$$e^{(k)} = x^{(k)} - x^*$$

denote the error at iteration k.

Since

$$x^{(k+1)} = Tx^{(k)} + c$$

and

$$x^* = Tx^* + c,$$

subtracting the two equations gives

$$e^{(k+1)} = Te^{(k)}.$$

Repeated application yields

$$\boxed{e^{(k)} = T^k e^{(0)}.} \tag{11}$$

Thus, the convergence problem reduces to understanding the behavior of the matrix powers

$$T,\ T^2,\ T^3, \ldots.$$

A fundamental result from numerical linear algebra states that

$$T^k \longrightarrow 0 \quad \text{as } k \to \infty$$

if and only if

$$\boxed{\rho(T) < 1,}$$

where $\rho(T)$ denotes the spectral radius of T:

$$\rho(T) = \max_i |\lambda_i(T)|.$$

Consequently, a stationary iteration converges to the unique solution for every initial guess if and only if the spectral radius of its iteration matrix is strictly less than one.

This provides the mathematical foundation for studying and comparing iterative methods.

However, knowing that an iteration converges is only part of the problem. In computational science, we are equally interested in how quickly it converges.

The Gauss–Seidel method improves upon Jacobi for many important classes of matrices, but its convergence can still be extremely slow.

Can we modify Gauss–Seidel to obtain a faster method without fundamentally changing its simple computational structure?

The answer leads us to the successive over-relaxation method, which we investigate in the next section.

A Connection to Optimization and Physical Equilibrium

For the more advanced reader who may still be wondering why such iterative procedures should work in the first place, there is another interesting way to look at the problem. Suppose that A is a symmetric positive definite matrix. Then solving

$$Ax = b$$

is equivalent to minimizing the quadratic functional

$$\boxed{E(x) = \frac{1}{2}x^T Ax - x^T b.}$$

Indeed, its gradient is

$$\nabla E(x) = Ax - b,$$

and therefore the stationary point satisfies

$$\nabla E(x) = 0 \quad \Longleftrightarrow \quad Ax = b.$$

Moreover, since the Hessian is

$$\nabla^2 E(x) = A,$$

and A is positive definite, this stationary point is the unique global minimum.

This interpretation is closely related to the principle of minimum potential energy in mechanics. Many stable physical systems, when displaced from equilibrium, tend to evolve toward a state of lower energy. In a certain sense, nature is already performing its own optimization! We may think of an iterative numerical method in a similar spirit: starting from an initial approximation, we repeatedly correct the current state in the hope of approaching the desired equilibrium. Some methods make rapid progress, others move painfully slowly, and some may fail to converge altogether. Of course, not every iterative method follows a strictly decreasing energy path, and the analogy should not be taken as a general convergence proof. Nevertheless, it provides a useful intuition for the problem we are studying. Our task is not merely to move the approximation toward the solution, but to design a procedure that makes this journey as smooth, reliable, and

swift as possible. As we shall see, the relaxation parameter in the SOR method gives us precisely one way to influence that journey.

One might object that our energy interpretation applies only when A is symmetric positive definite. That is certainly true. However, there is some good news: many important systems arising from partial differential equations, particularly those obtained by discretizing self-adjoint elliptic operators, naturally possess this structure. For a general matrix, the situation is somewhat more complicated, but we are not entirely without alternatives. For example, if A is nonsingular, we may transform $Ax = b$ into the equivalent symmetric positive definite system

$$A^T Ax = A^T b.$$

We may also consider suitable regularization techniques when the problem is ill-conditioned or requires additional stabilization. Of course, these transformations are not without computational consequences: forming $A^T A$ squares the condition number in the Euclidean norm and may destroy useful sparsity, while regularization generally modifies the original problem. Nevertheless, the essential idea remains valuable. Even when the original system does not admit a simple minimum-energy interpretation, optimization and matrix transformations often provide alternative ways of studying and solving it. We shall not pursue these generalizations here, since our principal interest lies in the classical SOR method and the structured matrices for which its optimal relaxation parameter can be determined analytically.

3 The Successive Over-Relaxation Method

Before introducing the successive over-relaxation method mathematically, let us return to a simple geometric picture.

Suppose someone is standing at the origin $(0, 0)$ in the Cartesian plane and wishes to reach the point $(1, 1)$. They may begin by choosing a direction, taking a step, examining their new position, and then calculating the next step. By repeating this procedure systematically, they hope eventually to reach their destination.

This is roughly the geometric intuition behind the Jacobi and Gauss–Seidel methods. We begin with an initial approximation, calculate a new approximation according to a prescribed rule, and repeat the process in the hope of approaching the exact solution.

But must we always accept the newly calculated position exactly as it is?

Suppose our current position is represented by the vector $x^{(k)}$, and an iterative procedure suggests a new position $\widetilde{x}^{(k+1)}$. Instead of moving directly to that position, we might decide to travel only 80% of the suggested displacement. In that case, our next position would be

$$x^{(k+1)} = 0.2x^{(k)} + 0.8\widetilde{x}^{(k+1)}.$$

In other words, we retain 20% of the previous approximation and accept 80% of the newly calculated one.

There is a familiar intuition behind this idea. The reader might remember going on a trek during their middle school years. When following an unfamiliar trail, one does not always rush toward the next visible point. It is often useful to look ahead while also keeping track of the path already traveled. Moving too cautiously may slow the journey, but moving too aggressively may lead us away from the intended route.

A similar consideration arises in iterative numerical methods. We have a current approximation and a newly calculated correction. How much of that correction should we accept?

Sometimes taking a small corrective step improves stability. Sometimes it merely slows convergence. And, perhaps surprisingly, sometimes moving *beyond* the newly calculated approximation, while still retaining a contribution from the previous approximation, can bring us to the solution considerably faster. This is precisely the intuition behind the successive over-relaxation (SOR) method.

Of course, the trekking analogy is only an intuition. An iterative method does not possess a physical sense of direction, and moving farther does not necessarily mean moving closer to the solution. Whether a particular choice accelerates convergence must ultimately be determined through mathematical analysis.

3.1 Introducing the Relaxation Parameter

Suppose that at iteration k we have the current approximation $x^{(k)}$, and a given iterative procedure produces a candidate approximation $\widetilde{x}^{(k+1)}$.

The corresponding correction is

$$d^{(k)} = \widetilde{x}^{(k+1)} - x^{(k)}.$$

Rather than accepting the entire correction, we introduce a scalar parameter ω and define

$$x^{(k+1)} = x^{(k)} + \omega d^{(k)}. \tag{12}$$

Substituting the definition of $d^{(k)}$ gives

$$x^{(k+1)} = x^{(k)} + \omega \left(\widetilde{x}^{(k+1)} - x^{(k)} \right).$$

Rearranging,

$$\boxed{x^{(k+1)} = (1-\omega)x^{(k)} + \omega \widetilde{x}^{(k+1)}.} \tag{13}$$

The scalar ω is called the *relaxation parameter*.

Three cases are particularly important.

1. **Under-relaxation:** $0 < \omega < 1$**.**

   We accept only a fraction of the suggested correction. The new approximation lies between the previous approximation and the candidate approximation.

2. **No relaxation:** $\omega = 1$**.**

   We accept the candidate approximation without modification:

   $$x^{(k+1)} = \widetilde{x}^{(k+1)}.$$

3. **Over-relaxation:** $\omega > 1$**.**

   We move beyond the candidate approximation in the direction of the suggested correction. This may accelerate convergence, although an excessively large parameter may cause divergence.

For example, suppose the current position is

$$x^{(k)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

and the candidate position is

$$\widetilde{x}^{(k+1)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Then the relaxed update becomes

$$x^{(k+1)} = \omega \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Thus,

ω	$x^{(k+1)}$
0.5	$(0.5, 0.5)$
0.8	$(0.8, 0.8)$
1.0	$(1.0, 1.0)$
1.5	$(1.5, 1.5)$

Geometrically, the relaxation parameter controls the distance traveled along the proposed correction direction.

However, our ultimate objective is not simply to travel farther or shorter distances. It is to choose ω so that the resulting sequence converges to the solution as efficiently as possible.

3.2 Successive Over-Relaxation as a Modification of Gauss–Seidel

We now apply the relaxation idea to the Gauss–Seidel method.

Recall that for a system

$$Ax = b,$$

the Gauss–Seidel update for the ith component is

$$\widetilde{x}_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right). \tag{14}$$

Here, $\widetilde{x}_i^{(k+1)}$ denotes the candidate value obtained from the Gauss–Seidel formula.

The essential feature is that the most recently updated components are used immediately.

Instead of accepting this candidate value directly, we apply the relaxation formula

$$x_i^{(k+1)} = (1-\omega) x_i^{(k)} + \omega \widetilde{x}_i^{(k+1)}.$$

Substituting (14), we obtain

$$\boxed{\begin{aligned} x_i^{(k+1)} &= (1-\omega) x_i^{(k)} \\ &\quad + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right). \end{aligned}} \tag{15}$$

This is the *successive over-relaxation (SOR) method*.

Notice that when $\omega = 1$, the first term disappears and we recover the ordinary Gauss–Seidel method.

Thus, Gauss–Seidel is a special case of SOR.

The parameter ω provides an additional degree of freedom that allows us to modify the convergence behavior without fundamentally changing the computational structure of Gauss–Seidel.

3.3 A Simple Numerical Example

Let us return to the system

$$\begin{aligned} 4x + y &= 9, \\ x + 3y &= 7. \end{aligned}$$

Recall that the exact solution is

$$x^* = \frac{20}{11}, \qquad y^* = \frac{19}{11}.$$

The SOR iteration becomes

$$\begin{aligned} x^{(k+1)} &= (1-\omega)x^{(k)} + \frac{\omega}{4}\left(9 - y^{(k)}\right), \\ y^{(k+1)} &= (1-\omega)y^{(k)} + \frac{\omega}{3}\left(7 - x^{(k+1)}\right). \end{aligned} \tag{16}$$

Starting from

$$x^{(0)} = 0, \qquad y^{(0)} = 0,$$

suppose we choose

$$\omega = 1.2.$$

The first component becomes

$$\begin{aligned} x^{(1)} &= (1 - 1.2)(0) + \frac{1.2}{4}(9 - 0) \\ &= 2.7. \end{aligned}$$

Using this newly computed value, the second component becomes

$$\begin{aligned} y^{(1)} &= (1 - 1.2)(0) + \frac{1.2}{3}(7 - 2.7) \\ &= 1.72. \end{aligned}$$

Thus, after the first iteration,

$$\boxed{\begin{bmatrix} x^{(1)} \\ y^{(1)} \end{bmatrix} = \begin{bmatrix} 2.7 \\ 1.72 \end{bmatrix}.}$$

Notice that the first component has moved beyond the ordinary Gauss–Seidel candidate 2.25.

This is precisely the effect of over-relaxation.

However, the value 1.2 was chosen only for illustration. It is not necessarily the optimal relaxation parameter for this system.

Determining an optimal parameter requires us to understand how ω influences the iteration matrix and its eigenvalues.

3.4 Matrix Formulation of SOR

We now derive the matrix form of the SOR iteration.

As before, write

$$A = D + L + U,$$

where D, L, and U denote the diagonal, strictly lower triangular, and strictly upper triangular parts of A, respectively.

Starting from the componentwise SOR formula (15), multiply both sides by a_{ii}:

$$\begin{aligned} a_{ii}x_i^{(k+1)} = (1-\omega)a_{ii}x_i^{(k)} + \omega b_i \\ - \omega \sum_{j<i} a_{ij}x_j^{(k+1)} - \omega \sum_{j>i} a_{ij}x_j^{(k)}. \end{aligned}$$

Move the terms involving the newly computed components to the left-hand side:

$$\begin{aligned} a_{ii}x_i^{(k+1)} + \omega \sum_{j<i} a_{ij}x_j^{(k+1)} = (1-\omega)a_{ii}x_i^{(k)} \\ - \omega \sum_{j>i} a_{ij}x_j^{(k)} + \omega b_i. \end{aligned}$$

Collecting these equations for all components gives

$$(D + \omega L)x^{(k+1)} = \left[(1-\omega)D - \omega U\right] x^{(k)} + \omega b. \tag{17}$$

Since $D + \omega L$ is lower triangular with nonzero diagonal entries, it is invertible.

Therefore,

$$\boxed{\begin{aligned} x^{(k+1)} = (D + \omega L)^{-1} \left[(1-\omega)D - \omega U\right] x^{(k)} \\ + \omega (D + \omega L)^{-1} b. \end{aligned}} \tag{18}$$

Define the SOR iteration matrix

$$\boxed{T_\omega = (D + \omega L)^{-1} \left[(1-\omega)D - \omega U\right].} \tag{19}$$

Also, define

$$c_\omega = \omega (D + \omega L)^{-1} b.$$

The SOR method now takes the familiar stationary iteration form

$$\boxed{x^{(k+1)} = T_\omega x^{(k)} + c_\omega.} \tag{20}$$

At $\omega = 1$,

$$\begin{aligned} T_1 &= (D + L)^{-1} \left[(1-1)D - U\right] \\ &= -(D + L)^{-1} U. \end{aligned}$$

Thus,

$$\boxed{T_1 = T_{GS},}$$

confirming once again that Gauss–Seidel is the special case of SOR corresponding to $\omega = 1$.

We have now introduced a family of iteration matrices indexed by the relaxation parameter ω.

This leads to the central mathematical question of our investigation:

For which values of ω does the iteration converge, and which value makes convergence fastest?

Since the error satisfies

$$e^{(k+1)} = T_\omega e^{(k)},$$

we obtain

$$e^{(k)} = T_\omega^k e^{(0)}.$$

Consequently, the SOR method converges for every initial approximation precisely when

$$\boxed{\rho(T_\omega) < 1.}$$

Moreover, the spectral radius governs the asymptotic convergence factor.

Our objective is therefore to determine, whenever possible, a relaxation parameter satisfying

$$\boxed{\omega_{\text{opt}} \in \arg\min_{\omega>0} \rho(T_\omega).}$$

The existence, uniqueness, and characterization of such a parameter depend on the properties of the coefficient matrix.

In the following section, we investigate these questions and derive an explicit optimal relaxation parameter for an important class of matrices arising in numerical solutions of differential equations.

4 Convergence Analysis and Optimal Relaxation Parameters

We have seen that the successive over-relaxation method introduces a parameter ω into the Gauss–Seidel iteration, allowing us to control the magnitude of each successive correction.

However, introducing an additional parameter does not automatically improve an iterative method. A poor choice may slow convergence considerably or even cause an otherwise convergent iteration to diverge.

This raises two fundamental questions:

1. For which values of ω does the SOR method converge?
2. Among the admissible values, which choice of ω produces the fastest convergence?

The first question concerns the stability and convergence of the iteration. The second concerns its efficiency.

Recall that the SOR iteration matrix is

$$T_\omega = (D + \omega L)^{-1} \left[(1-\omega)D - \omega U\right]. \tag{21}$$

The error after k iterations satisfies

$$e^{(k)} = T_\omega^k e^{(0)}.$$

Consequently, the iteration converges for every initial approximation if and only if

$$\boxed{\rho(T_\omega) < 1.} \tag{22}$$

Our objective is therefore to understand how the spectral radius of T_ω depends on ω.

4.1 A Necessary Condition for SOR Convergence

Before examining any particular coefficient matrix, we can establish a necessary condition that applies to every SOR iteration with a well-defined splitting.

Theorem 4.1. *Let $A \in \mathbb{R}^{n\times n}$ have nonzero diagonal entries, and let*

$$A = D + L + U.$$

If the SOR method converges for every initial approximation, then

$$\boxed{0 < \omega < 2.}$$

Proof. Recall that

$$T_\omega = (D + \omega L)^{-1} \left[(1-\omega)D - \omega U\right].$$

Taking determinants, we obtain

$$\det(T_\omega) = \frac{\det\left((1-\omega)D - \omega U\right)}{\det(D + \omega L)}.$$

Now observe that $D + \omega L$ is lower triangular, while $(1-\omega)D - \omega U$ is upper triangular.

Since the determinant of a triangular matrix equals the product of its diagonal entries,

$$\det(D + \omega L) = \prod_{i=1}^{n} a_{ii}.$$

Similarly,

$$\begin{aligned}\det\left((1-\omega)D - \omega U\right) &= \prod_{i=1}^{n}(1-\omega)a_{ii} \\ &= (1-\omega)^n \prod_{i=1}^{n} a_{ii}.\end{aligned}$$

Therefore,

$$\boxed{\det(T_\omega) = (1-\omega)^n.} \tag{23}$$

Let $\lambda_1, \ldots, \lambda_n$ denote the eigenvalues of T_ω, counted with algebraic multiplicity.

We know that the determinant equals the product of the eigenvalues:

$$\det(T_\omega) = \prod_{i=1}^{n} \lambda_i.$$

Consequently,

$$|1-\omega|^n = \prod_{i=1}^{n} |\lambda_i|.$$

By the definition of spectral radius,

$$|\lambda_i| \le \rho(T_\omega)$$

for every i.

Hence,

$$|1-\omega|^n \le \rho(T_\omega)^n.$$

Taking the nth root gives

$$\boxed{\rho(T_\omega) \geq |1-\omega|.} \tag{24}$$

For convergence, we require

$$\rho(T_\omega) < 1.$$

Therefore,

$$|1-\omega| < 1,$$

which is equivalent to

$$-1 < 1-\omega < 1.$$

Rearranging,

$$\boxed{0 < \omega < 2.}$$

This proves the necessary condition. □

Remark 4.2. The condition $0 < \omega < 2$ is necessary but not sufficient for convergence of SOR for an arbitrary coefficient matrix.

In other words, choosing ω inside this interval does not guarantee convergence unless additional assumptions are imposed on A.

However, for symmetric positive definite matrices, the situation is considerably more favorable: SOR converges for every relaxation parameter satisfying $0 < \omega < 2$.

We shall establish this result in the next subsection.

4.2 An Immediate Consequence for Optimal Relaxation

The preceding theorem gives us more than a restriction on the admissible relaxation parameter.

It also establishes the lower bound

$$\boxed{\rho(T_\omega) \geq |1-\omega|.}$$

This observation is important because our objective is to minimize the spectral radius.

In particular, for $\omega > 1$,

$$\rho(T_\omega) \geq \omega - 1.$$

Thus, increasing ω cannot improve convergence indefinitely. As ω approaches 2, this lower bound approaches 1.

For example,

ω	Lower bound for $\rho(T_\omega)$
1.2	0.2
1.5	0.5
1.8	0.8
1.9	0.9
1.99	0.99

These are lower bounds, not necessarily the actual spectral radii.

Nevertheless, they already reveal something interesting: although over-relaxation may accelerate Gauss–Seidel, choosing an excessively large relaxation parameter eventually becomes counterproductive.

The optimal parameter must balance the acceleration obtained through extrapolation against the possibility of introducing slowly decaying or oscillatory errors.

This leads us naturally to the next question:

Under what conditions can we guarantee convergence throughout the interval $0 < \omega < 2$*?*

4.3 Convergence of SOR for Symmetric Positive Definite Matrices

We have established that $0 < \omega < 2$ is a necessary condition for convergence of the SOR method. However, for a general matrix, this condition alone does not guarantee convergence.

Fortunately, for symmetric positive definite matrices, the situation is considerably more favorable. In fact, the same condition becomes both necessary and sufficient.

Before proving this result, let us return to the quadratic energy interpretation introduced earlier.

Suppose that A is symmetric positive definite. Then the linear system

$$Ax = b$$

is equivalent to minimizing

$$E(x) = \frac{1}{2}x^T Ax - b^T x. \tag{25}$$

Since

$$\nabla E(x) = Ax - b$$

and

$$\nabla^2 E(x) = A,$$

the functional E has a unique global minimizer $x^* = A^{-1}b$.

Now consider what happens when we change only one component of the current approximation.

Let e_i denote the ith standard basis vector and consider the point

$$x + \alpha e_i,$$

where α represents a correction to the ith component.

Substituting into the energy functional gives

$$E(x + \alpha e_i) = \frac{1}{2}(x + \alpha e_i)^T A(x + \alpha e_i) - b^T(x + \alpha e_i).$$

Expanding and using the symmetry of A, we obtain

$$\begin{aligned} E(x + \alpha e_i) = \frac{1}{2}x^T Ax - b^T x \\ + \alpha e_i^T(Ax - b) + \frac{1}{2}\alpha^2 e_i^T A e_i. \end{aligned}$$

Therefore,

$$\boxed{E(x + \alpha e_i) - E(x) = \alpha(Ax - b)_i + \frac{1}{2}a_{ii}\alpha^2.} \tag{26}$$

Since A is positive definite,

$$a_{ii} > 0.$$

Thus, the right-hand side is a strictly convex quadratic function of α.

Differentiating with respect to α,

$$\frac{d}{d\alpha}\left[E(x+\alpha e_i)-E(x)\right]=(Ax-b)_i+a_{ii}\alpha.$$

Setting the derivative equal to zero gives

$$\boxed{\alpha_*=-\frac{(Ax-b)_i}{a_{ii}}.}$$

This is precisely the correction obtained by minimizing the energy with respect to the ith coordinate while keeping the other coordinates fixed.

When the coordinates are updated successively, using the most recently available values, we recover the Gauss–Seidel method.

Hence, for symmetric positive definite matrices, Gauss–Seidel can be interpreted as a sequence of exact coordinatewise energy minimizations.

But what happens if we introduce relaxation?

Instead of taking the correction α_*, SOR takes

$$\alpha=\omega\alpha_*.$$

Substituting into (26), we obtain

$$E(x+\omega\alpha_* e_i)-E(x)=\omega\alpha_*(Ax-b)_i+\frac{1}{2}a_{ii}\omega^2\alpha_*^2.$$

Since

$$(Ax-b)_i=-a_{ii}\alpha_*,$$

we obtain

$$\begin{aligned}E(x+\omega\alpha_* e_i)-E(x)&=-\omega a_{ii}\alpha_*^2+\frac{1}{2}\omega^2 a_{ii}\alpha_*^2\\&=-\frac{1}{2}\omega(2-\omega)a_{ii}\alpha_*^2.\end{aligned}$$

Therefore,

$$\boxed{E(x+\omega\alpha_* e_i)-E(x)=-\frac{\omega(2-\omega)}{2}a_{ii}\alpha_*^2.} \tag{27}$$

This identity is particularly revealing.

Since $a_{ii}>0$, the energy decreases whenever

$$\omega(2-\omega)>0$$

and the coordinate correction is nonzero.

But

$$\omega(2-\omega)>0 \quad\Longleftrightarrow\quad 0<\omega<2.$$

Thus, for every $0<\omega<2$, each SOR coordinate update decreases the energy unless that coordinate already requires no correction.

Notice that even when $\omega > 1$, so that the method moves beyond the coordinatewise minimizer, the energy still decreases as long as $\omega < 2$.

This gives a geometric explanation for the convergence interval that we previously obtained using determinants.

Theorem 4.3 (Convergence of SOR for SPD Matrices)**.** *Let $A \in \mathbb{R}^{n\times n}$ be symmetric positive definite. Then the successive over-relaxation method converges to the unique solution of $Ax = b$, for every initial approximation, if and only if*

$$\boxed{0 < \omega < 2.}$$

Proof. We have already proved that $0 < \omega < 2$ is necessary for convergence.

It remains to prove sufficiency.

Let x^* denote the unique solution of $Ax = b$, and define the error

$$e = x - x^*.$$

Since $Ax^* = b$, expanding the energy around x^* gives

$$E(x) - E(x^*) = \frac{1}{2}(x - x^*)^T A(x - x^*).$$

Therefore,

$$\boxed{E(x) - E(x^*) = \frac{1}{2}e^T Ae.} \tag{28}$$

Because A is symmetric positive definite, the expression

$$\|e\|_A = \sqrt{e^T Ae}$$

defines a norm.

Now consider one complete SOR iteration, which consists of updating all n coordinates successively.

By (27), each coordinate update decreases the energy or leaves it unchanged.

Moreover, if every coordinate update leaves the energy unchanged, then every coordinate correction must be zero.

In that case, the current approximation satisfies every equation of $Ax = b$ and therefore equals x^*.

Consequently, for any $x \neq x^*$, one complete SOR iteration strictly decreases the energy:

$$E(x^{(k+1)}) < E(x^{(k)}).$$

Using (28), this gives

$$\|e^{(k+1)}\|_A < \|e^{(k)}\|_A$$

whenever $e^{(k)} \neq 0$.

Recall that SOR is a stationary iteration, so its errors satisfy

$$e^{(k+1)} = T_\omega e^{(k)}.$$

Thus,

$$\|T_\omega e\|_A < \|e\|_A \qquad \text{for every } e \neq 0.$$

Since T_ω is linear and the unit sphere

$$S = \{e \in \mathbb{R}^n : \|e\|_A = 1\}$$

is compact, the continuous function

$$e \longmapsto \|T_\omega e\|_A$$

attains its maximum on S.

The strict inequality above implies that this maximum is some constant $q < 1$.

Hence,

$$\|T_\omega e\|_A \leq q\|e\|_A$$

for every $e \in \mathbb{R}^n$.

Applying this inequality repeatedly,

$$\|e^{(k)}\|_A \leq q^k \|e^{(0)}\|_A.$$

Since $0 \leq q < 1$,

$$\lim_{k\to\infty} \|e^{(k)}\|_A = 0.$$

Therefore,

$$\boxed{\lim_{k\to\infty} x^{(k)} = x^*.}$$

Thus, SOR converges for every initial approximation whenever $0 < \omega < 2$. □

Remark 4.4. The preceding proof establishes convergence, but it does not determine the optimal relaxation parameter.

Indeed, the energy decrease in (27) is largest for an individual coordinate update when $\omega = 1$, corresponding to Gauss–Seidel.

However, minimizing the energy during one coordinate update is not the same as minimizing the number of complete iterations required to solve the system.

By moving beyond the coordinatewise minimizer, SOR may produce a sequence of approximations that converges substantially faster in the long run.

This is why over-relaxation can outperform ordinary Gauss–Seidel even though each individual Gauss–Seidel coordinate update minimizes the energy along its coordinate.

5 Derivation of the Optimal SOR Parameter

We have established that the SOR method converges for every $0 < \omega < 2$ when the coefficient matrix is symmetric positive definite.

However, convergence is only part of our objective. We are particularly interested in choosing a relaxation parameter that makes the iteration converge as rapidly as possible.

Since the asymptotic convergence factor is governed by the spectral radius, our problem is to determine

$$\boxed{\omega_{\text{opt}} \in \arg\min_{0<\omega<2} \rho(T_\omega).} \tag{29}$$

For a general matrix, this optimization problem may be difficult to solve analytically.

Fortunately, for certain structured matrices, the eigenvalues of the SOR iteration matrix are related to those of the Jacobi iteration matrix through a remarkable algebraic identity.

We shall derive this relationship first and then use it to determine the optimal relaxation parameter.

5.1 The Relationship Between Jacobi and SOR Eigenvalues

Consider a nonsingular tridiagonal matrix

$$A = \begin{bmatrix} a_1 & c_1 & 0 & \cdots & 0 \\ b_1 & a_2 & c_2 & \ddots & \vdots \\ 0 & b_2 & a_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & c_{n-1} \\ 0 & \cdots & 0 & b_{n-1} & a_n \end{bmatrix},$$

where every diagonal entry a_i is nonzero.

As before, write

$$A = D + L + U.$$

The Jacobi iteration matrix is

$$T_J = -D^{-1}(L + U), \tag{30}$$

while the SOR iteration matrix is

$$T_\omega = (D + \omega L)^{-1} \left[(1 - \omega)D - \omega U\right]. \tag{31}$$

Our objective is to establish a relationship between the eigenvalues of these two matrices.

Let λ be an eigenvalue of T_ω. Then

$$\det(\lambda I - T_\omega) = 0.$$

Substituting the expression for T_ω, we obtain

$$\det\left[\lambda I - (D + \omega L)^{-1}\left((1 - \omega)D - \omega U\right)\right] = 0.$$

Multiplying the matrix inside the determinant on the left by $D + \omega L$, which is invertible, we obtain the equivalent condition

$$\det\left[\lambda(D + \omega L) - \left((1 - \omega)D - \omega U\right)\right] = 0.$$

Expanding,

$$\det\left[\lambda D + \lambda\omega L - (1 - \omega)D + \omega U\right] = 0.$$

Collecting the diagonal terms gives

$$\boxed{\det\left[(\lambda + \omega - 1)D + \lambda\omega L + \omega U\right] = 0.} \tag{32}$$

At first sight, this expression does not seem particularly helpful. The coefficients of L and U are different, and the Jacobi matrix contains their sum $L + U$.

But here the tridiagonal structure of A plays an important role.

Suppose $\lambda \neq 0$, and choose a complex number s satisfying

$$s^2 = \lambda.$$

Consider the invertible diagonal matrix

$$S = \mathrm{diag}(1, s, s^2, \ldots, s^{n-1}).$$

Because L contains nonzero entries only immediately below the diagonal, and U contains nonzero entries only immediately above it, we can examine the similarity transformation entry by entry.

Recall that

$$S = \mathrm{diag}(1, s, s^2, \ldots, s^{n-1}),$$

and therefore

$$S^{-1} = \mathrm{diag}(1, s^{-1}, s^{-2}, \ldots, s^{-(n-1)}).$$

Since both S and S^{-1} are diagonal, for any matrix M we have

$$(S^{-1}MS)_{ij} = (S^{-1})_{ii} M_{ij} S_{jj} = s^{-(i-1)} M_{ij} s^{j-1}.$$

Thus,

$$\boxed{(S^{-1}MS)_{ij} = s^{j-i} M_{ij}.} \tag{33}$$

Now consider L. Its only potentially nonzero entries occur when $i = j + 1$. Hence,

$$(S^{-1}LS)_{ij} = s^{j-i} L_{ij} = s^{-1} L_{ij}.$$

All other entries of L are zero and remain zero. Consequently,

$$\boxed{S^{-1}LS = s^{-1}L.}$$

Similarly, the only potentially nonzero entries of U occur when $j = i + 1$. Therefore,

$$(S^{-1}US)_{ij} = s^{j-i} U_{ij} = sU_{ij}.$$

Hence,

$$\boxed{S^{-1}US = sU.}$$

Finally, since D and S are diagonal matrices, they commute, and therefore

$$\boxed{S^{-1}DS = D.}$$

Consequently,

$$\begin{aligned} &S^{-1}\left[(\lambda + \omega - 1)D + \lambda\omega L + \omega U\right] S \\ &= (\lambda + \omega - 1)D + \lambda\omega s^{-1} L + \omega s U. \end{aligned}$$

Using $\lambda = s^2$, we obtain

$$\lambda s^{-1} = s.$$

Therefore,

$$S^{-1}\left[(\lambda+\omega-1)D+\lambda\omega L+\omega U\right]S$$
$$=(\lambda+\omega-1)D+\omega s(L+U).$$

Since similar matrices have the same determinant, equation (32) becomes

$$\boxed{\det\left[(\lambda+\omega-1)D+\omega s(L+U)\right]=0.} \tag{34}$$

Now factor out ωsD:

$$\det\left[\omega sD\left(\frac{\lambda+\omega-1}{\omega s}I+D^{-1}(L+U)\right)\right]=0.$$

Since ωsD is invertible, we must have

$$\det\left[\frac{\lambda+\omega-1}{\omega s}I+D^{-1}(L+U)\right]=0.$$

Recall that

$$T_J=-D^{-1}(L+U).$$

Hence,

$$\det\left[\frac{\lambda+\omega-1}{\omega s}I-T_J\right]=0.$$

This means that

$$\mu=\frac{\lambda+\omega-1}{\omega s}$$

is an eigenvalue of the Jacobi iteration matrix.

Therefore,

$$\lambda+\omega-1=\omega\mu s.$$

Squaring both sides and recalling that $s^2=\lambda$, we obtain

$$\boxed{(\lambda+\omega-1)^2=\omega^2\mu^2\lambda.} \tag{35}$$

This is the fundamental relationship between the eigenvalues of the Jacobi and SOR iteration matrices for the tridiagonal systems under consideration.

The derivation assumed $\lambda\neq 0$. The zero-eigenvalue case can be treated separately: from

$$\det(T_\omega)=(1-\omega)^n,$$

we see that $\lambda=0$ can occur only when $\omega=1$, in which case the relationship also holds with $\mu=0$.

The significance of (35) is that it allows us to study the eigenvalues of SOR through those of Jacobi.

Rather than computing the spectrum of T_ω independently for every relaxation parameter, we can exploit the simpler structure of T_J.

In the next subsection, we shall use this identity to determine the value of ω that minimizes the spectral radius.

5.2 Derivation of the Optimal Relaxation Parameter

We have established the relationship

$$(\lambda + \omega - 1)^2 = \omega^2 \mu^2 \lambda, \tag{36}$$

where μ is an eigenvalue of the Jacobi iteration matrix and λ is an associated eigenvalue of the SOR iteration matrix.

Our objective is to determine the value of ω that minimizes the spectral radius of T_ω.

For the remainder of this derivation, we assume that A is symmetric positive definite and tridiagonal, with nonzero diagonal entries, and that the Jacobi iteration converges.

Consequently, the eigenvalues of T_J are real, and

$$r := \rho(T_J) < 1.$$

We shall see that the optimal relaxation parameter depends only on r.

A Quadratic Equation for the SOR Eigenvalues

Expanding (36), we obtain

$$\lambda^2 + 2(\omega - 1)\lambda + (\omega - 1)^2 = \omega^2 \mu^2 \lambda.$$

Collecting terms,

$$\boxed{\lambda^2 + \left[2(\omega - 1) - \omega^2 \mu^2\right] \lambda + (\omega - 1)^2 = 0.} \tag{37}$$

Thus, the eigenvalues of SOR are determined by a quadratic equation involving the relaxation parameter ω and the Jacobi eigenvalues μ.

Recall from elementary algebra that the nature of the roots of a quadratic equation is determined by its discriminant.

For (37), the discriminant is

$$\begin{aligned} \Delta &= \left[2(\omega - 1) - \omega^2 \mu^2\right]^2 - 4(\omega - 1)^2 \\ &= \omega^4 \mu^4 - 4\omega^2 \mu^2 (\omega - 1). \end{aligned}$$

Factoring,

$$\boxed{\Delta = \omega^2 \mu^2 \left[\omega^2 \mu^2 - 4(\omega - 1)\right].} \tag{38}$$

This expression is the key to determining the optimal relaxation parameter.

What Happens When the Discriminant Changes Sign?

Consider first the case $\Delta > 0$.

The quadratic equation has two distinct real roots. Their magnitudes depend on ω and μ, and one of them may dominate the convergence behavior. To see why, consider a Jacobi eigenvalue satisfying $|\mu| = r = \rho(T_J)$. For $0 < \omega < \omega_*$, where ω_* denotes the transition parameter to be

determined shortly, the corresponding discriminant is strictly positive. Solving the quadratic equation gives two distinct real SOR eigenvalues:

$$\lambda_{\pm} = \frac{\omega^2 r^2 - 2(\omega - 1) \pm \omega r \sqrt{\omega^2 r^2 - 4(\omega - 1)}}{2}.$$

The larger eigenvalue λ_+ is positive and provides a lower bound for the spectral radius:

$$\rho(T_\omega) \geq \lambda_+.$$

As we shall prove later, this eigenvalue decreases as ω approaches ω_* from below. At the transition, the two real eigenvalues coincide, and their common value is $\omega_* - 1$. Beyond this point, the eigenvalues associated with $|\mu| = r$ become complex conjugates, and their common magnitude increases with ω. Thus, the positive-discriminant case explains why increasing ω initially improves convergence, while the negative-discriminant case explains why increasing it beyond the transition eventually becomes counterproductive. Together, these two behaviors lead us to the optimal relaxation parameter.

Now suppose $\Delta < 0$.

The roots become a complex conjugate pair, say λ_1 and λ_2.

By Vieta's formula,

$$\lambda_1 \lambda_2 = (\omega - 1)^2.$$

Since the roots are complex conjugates,

$$\lambda_2 = \overline{\lambda_1}.$$

Therefore,

$$|\lambda_1|^2 = (\omega - 1)^2,$$

and hence

$$\boxed{|\lambda_1| = |\lambda_2| = |\omega - 1|.} \tag{39}$$

This is particularly interesting.

Recall that we previously established the universal lower bound

$$\rho(T_\omega) \geq |\omega - 1|.$$

Thus, when the relevant SOR eigenvalues form complex conjugate pairs, their magnitudes attain this lower bound.

However, we must consider all Jacobi eigenvalues, not merely one of them.

The largest magnitude of a Jacobi eigenvalue is

$$r = \rho(T_J).$$

For $\omega > 1$, the expression

$$\omega^2 \mu^2 - 4(\omega - 1)$$

is largest when $|\mu| = r$.

Consequently, the transition at which the discriminant becomes nonpositive for every Jacobi eigenvalue is determined by

$$\boxed{\omega^2 r^2 - 4(\omega - 1) = 0.} \tag{40}$$

At this transition, the two roots associated with $|\mu| = r$ coincide.

We shall now solve this equation.

Solving for the Optimal Parameter

Rearranging (40), we obtain

$$r^2\omega^2 - 4\omega + 4 = 0.$$

Applying the quadratic formula,

$$\omega = \frac{4 \pm \sqrt{16 - 16r^2}}{2r^2}.$$

Simplifying,

$$\omega = \frac{2\left(1 \pm \sqrt{1 - r^2}\right)}{r^2}.$$

Since

$$r^2 = \left(1 - \sqrt{1 - r^2}\right)\left(1 + \sqrt{1 - r^2}\right),$$

the two solutions can be written as

$$\omega_- = \frac{2}{1 + \sqrt{1 - r^2}},$$

and

$$\omega_+ = \frac{2}{1 - \sqrt{1 - r^2}}.$$

For $0 < r < 1$, the second solution satisfies $\omega_+ > 2$.

But we already know that SOR can converge only when

$$0 < \omega < 2.$$

Therefore, the admissible transition parameter is

$$\boxed{\omega_* = \frac{2}{1 + \sqrt{1 - r^2}}.} \tag{41}$$

Notice that

$$1 < \omega_* < 2.$$

Thus, the transition occurs in the over-relaxation region.

We still need to verify that this transition actually minimizes the spectral radius.

Why Is This Parameter Optimal?

We have obtained the candidate

$$\omega_* = \frac{2}{1 + \sqrt{1 - r^2}}, \qquad r = \rho(T_J), \quad 0 < r < 1.$$

However, finding a parameter by setting the discriminant equal to zero does not automatically prove that the parameter is optimal.

To establish optimality, we must show that

$$\rho(T_\omega) > \rho(T_{\omega_*})$$

for every $\omega \neq \omega_*$ in the convergence interval $0 < \omega < 2$.

We shall examine two cases.

Case 1: $\omega_* \leq \omega < 2$**.**

Recall that the discriminant of the quadratic equation for the SOR eigenvalues is

$$\Delta = \omega^2\mu^2 \left[\omega^2\mu^2 - 4(\omega - 1)\right].$$

Since every Jacobi eigenvalue satisfies $|\mu| \leq r$, we have

$$\omega^2\mu^2 - 4(\omega - 1) \leq \omega^2 r^2 - 4(\omega - 1).$$

The right-hand side vanishes at $\omega = \omega_*$ and is negative for $\omega_* < \omega < 2$. Therefore,

$$\boxed{\Delta \leq 0}$$

for every Jacobi eigenvalue throughout this interval.

When $\Delta < 0$, the two roots of the quadratic equation form a complex conjugate pair, say λ_1 and λ_2.

By Vieta's formula,

$$\lambda_1\lambda_2 = (\omega - 1)^2.$$

Since $\lambda_2 = \overline{\lambda_1}$,

$$|\lambda_1|^2 = (\omega - 1)^2.$$

Thus,

$$|\lambda_1| = |\lambda_2| = \omega - 1.$$

The same conclusion holds when $\Delta = 0$, because the repeated root is then $\lambda = \omega - 1$.

Consequently, every SOR eigenvalue has magnitude $\omega - 1$, and hence

$$\boxed{\rho(T_\omega) = \omega - 1, \qquad \omega_* \leq \omega < 2.} \tag{42}$$

But $\omega - 1$ is strictly increasing as a function of ω.

Therefore,

$$\rho(T_\omega) > \rho(T_{\omega_*}) \qquad \text{whenever } \omega > \omega_*.$$

We have proved that choosing a parameter larger than ω_* cannot improve the asymptotic convergence factor.

Case 2: $0 < \omega < \omega_*$**.**

Choose a Jacobi eigenvalue satisfying

$$|\mu| = r.$$

For this eigenvalue, the relationship between Jacobi and SOR becomes

$$(\lambda + \omega - 1)^2 = \omega^2 r^2 \lambda.$$

We may write

$$\lambda + \omega - 1 = \omega r\sqrt{\lambda}$$

for the branch corresponding to the larger positive root.

Introducing

$$t = \sqrt{\lambda},$$

we obtain

$$t^2 + \omega - 1 = \omega r t,$$

or equivalently,

$$\boxed{t^2 - \omega r t + \omega - 1 = 0.} \tag{43}$$

Applying the quadratic formula gives

$$t_{\pm}(\omega) = \frac{\omega r \pm \sqrt{\omega^2 r^2 - 4(\omega - 1)}}{2}.$$

We shall concentrate on the larger root,

$$\boxed{t_{+}(\omega) = \frac{\omega r + \sqrt{\omega^2 r^2 - 4(\omega - 1)}}{2}.} \tag{44}$$

For $0 < \omega < \omega_*$, the expression under the square root is strictly positive. Thus, $t_+(\omega)$ is real and positive.

Since $\lambda = t_+(\omega)^2$ is an eigenvalue of the SOR iteration matrix, we have

$$\boxed{\rho(T_\omega) \geq t_+(\omega)^2.} \tag{45}$$

Therefore, it is sufficient to prove that $t_+(\omega)$ decreases as ω approaches ω_*.

Showing that $t_+(\omega)$ decreases.

Rather than differentiating the explicit square-root expression, we can differentiate the quadratic equation (43) implicitly.

Recall that

$$t_+^2 - \omega r t_+ + \omega - 1 = 0,$$

where t_+ is a function of ω.

Differentiating both sides with respect to ω,

$$2t_+ \frac{dt_+}{d\omega} - r t_+ - \omega r \frac{dt_+}{d\omega} + 1 = 0.$$

Collecting the derivative terms,

$$(2t_+ - \omega r)\frac{dt_+}{d\omega} = r t_+ - 1.$$

Hence,

$$\boxed{\frac{dt_+}{d\omega} = \frac{r t_+ - 1}{2t_+ - \omega r}.} \tag{46}$$

We now examine the signs of the numerator and denominator separately.

First, from (44),

$$2t_+ - \omega r = \sqrt{\omega^2 r^2 - 4(\omega - 1)}.$$

Since $0 < \omega < \omega_*$, the discriminant is strictly positive. Therefore,

$$\boxed{2t_+ - \omega r > 0.}$$

Next, we must show that

$$rt_+ - 1 < 0.$$

To establish this, consider the quadratic polynomial

$$p(t) = t^2 - \omega r t + \omega - 1.$$

At $t = 1$,

$$\begin{aligned} p(1) &= 1 - \omega r + \omega - 1 \\ &= \omega(1 - r) > 0, \end{aligned}$$

because $0 < r < 1$.

Moreover, the vertex of this quadratic occurs at

$$t = \frac{\omega r}{2} < 1,$$

since $0 < \omega < \omega_* < 2$.

The quadratic opens upward and has two real roots. Since its vertex lies to the left of 1 and $p(1) > 0$, both roots must lie to the left of 1.

In particular,

$$\boxed{0 < t_+ < 1.}$$

Since $0 < r < 1$, it follows that

$$rt_+ < 1,$$

and therefore

$$\boxed{rt_+ - 1 < 0.}$$

Returning to (46), we have

$$\frac{dt_+}{d\omega} = \frac{\underbrace{rt_+ - 1}_{<0}}{\underbrace{2t_+ - \omega r}_{>0}}.$$

Consequently,

$$\boxed{\frac{dt_+}{d\omega} < 0, \qquad 0 < \omega < \omega_*.} \tag{47}$$

Thus, $t_+(\omega)$ is strictly decreasing throughout the interval $(0, \omega_*)$.

Comparing the roots at the transition.

At $\omega = \omega_*$, the discriminant vanishes:

$$\omega_*^2 r^2 - 4(\omega_* - 1) = 0.$$

Therefore,

$$\omega_*^2 r^2 = 4(\omega_* - 1).$$

The two roots coincide at

$$t_+(\omega_*) = \frac{\omega_* r}{2}.$$

Squaring,

$$\begin{aligned} t_+(\omega_*)^2 &= \frac{\omega_*^2 r^2}{4} \\ &= \omega_* - 1. \end{aligned}$$

Hence,

$$\boxed{t_+(\omega_*)^2 = \omega_* - 1.} \tag{48}$$

Since $t_+(\omega)$ is strictly decreasing for $0 < \omega < \omega_*$, we have

$$t_+(\omega) > t_+(\omega_*).$$

Both quantities are positive, so

$$t_+(\omega)^2 > t_+(\omega_*)^2 = \omega_* - 1.$$

Combining this inequality with (45), we conclude that

$$\boxed{\rho(T_\omega) > \omega_* - 1, \qquad 0 < \omega < \omega_*.} \tag{49}$$

On the other hand, from (42),

$$\rho(T_{\omega_*}) = \omega_* - 1.$$

Therefore,

$$\rho(T_\omega) > \rho(T_{\omega_*}) \qquad \text{for every } 0 < \omega < \omega_*.$$

This completes the second case.

Conclusion.

We have proved that

$$\rho(T_\omega) > \rho(T_{\omega_*})$$

whenever

$$0 < \omega < 2, \qquad \omega \neq \omega_*.$$

Thus, the spectral radius attains its minimum at $\omega = \omega_*$.

Recalling that

$$r = \rho(T_J),$$

we finally obtain

$$\boxed{\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho(T_J)^2}}.} \tag{50}$$

Furthermore, the corresponding optimal spectral radius is

$$\boxed{\rho(T_{\omega_{\text{opt}}}) = \omega_{\text{opt}} - 1.} \tag{51}$$

This result provides an explicit connection between the convergence properties of the Jacobi method and the optimal relaxation parameter of SOR.

It is worth emphasizing that this formula does not hold for every symmetric positive definite matrix. Our derivation relies on the special eigenvalue relationship established for the tridiagonal systems under consideration.

For more general matrices, additional structural assumptions are required to obtain the same formula.

5.3 Optimal SOR Parameter for a Finite-Difference Matrix

We have established that, for the symmetric positive definite tridiagonal matrices under consideration, the optimal SOR relaxation parameter is

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho(T_J)^2}},$$

where T_J denotes the Jacobi iteration matrix.

The formula is theoretically useful, but it still requires knowledge of the spectral radius of T_J. For a general matrix, determining this quantity may itself be computationally expensive.

Fortunately, the matrices arising from the finite-difference discretization of certain boundary value problems possess a special structure that allows us to calculate their eigenvalues explicitly.

Construction of the Finite-Difference Matrix

Consider the boundary value problem

$$-u''(x) = f(x), \qquad 0 < x < 1,$$

subject to the homogeneous Dirichlet boundary conditions

$$u(0) = u(1) = 0.$$

Divide $[0, 1]$ into $n + 1$ equal subintervals, with grid spacing

$$h = \frac{1}{n+1}.$$

The interior grid points are

$$x_i = ih, \qquad i = 1, \ldots, n.$$

Using the central-difference approximation,

$$u''(x_i) \approx \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2},$$

we obtain

$$-u_{i-1} + 2u_i - u_{i+1} = h^2 f(x_i).$$

Thus, the finite-difference discretization produces the linear system

$$Au = b,$$

where

$$A = \begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \ddots & \vdots \\ 0 & -1 & 2 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & -1 \\ 0 & \cdots & 0 & -1 & 2 \end{pmatrix}_{n\times n},$$

and

$$b_i = h^2 f(x_i).$$

The matrix A is symmetric positive definite and tridiagonal. Hence, the optimal SOR parameter derived previously applies.

Eigenvalues of the Jacobi Iteration Matrix

For the finite-difference matrix,

$$D = 2I.$$

Recall that the Jacobi iteration matrix is

$$T_J = -D^{-1}(L + U).$$

Therefore,

$$T_J = \frac{1}{2}\begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 1 & 0 & 1 & \ddots & \vdots \\ 0 & 1 & 0 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & 1 \\ 0 & \cdots & 0 & 1 & 0 \end{pmatrix}.$$

We now determine the eigenvalues of T_J analytically.

Let

$$T_J v = \mu v,$$

where $v \neq 0$ is an eigenvector.

Writing this equation componentwise gives

$$\frac{1}{2}(v_{i-1} + v_{i+1}) = \mu v_i.$$

Introducing the boundary values

$$v_0 = v_{n+1} = 0,$$

we can express all n equations through the recurrence relation

$$v_{i-1} + v_{i+1} = 2\mu v_i, \qquad i = 1, \ldots, n. \tag{52}$$

This is a second-order linear difference equation, similar in structure to the differential equation from which the finite-difference matrix originated.

Solving the characteristic equation.

As with linear differential equations with constant coefficients, we look for solutions of the form

$$v_i = z^i,$$

where z is a nonzero constant to be determined.

Substituting into (52), we obtain

$$z^{i-1} + z^{i+1} = 2\mu z^i.$$

Dividing by z^{i-1} gives

$$1 + z^2 = 2\mu z.$$

Rearranging,

$$\boxed{z^2 - 2\mu z + 1 = 0.} \tag{53}$$

Applying the quadratic formula,

$$z_{1,2} = \frac{2\mu \pm \sqrt{4\mu^2 - 4}}{2}.$$

Factoring 4 inside the square root,

$$z_{1,2} = \frac{2\mu \pm 2\sqrt{\mu^2 - 1}}{2}.$$

Therefore,

$$\boxed{z_{1,2} = \mu \pm \sqrt{\mu^2 - 1}.} \tag{54}$$

Before proceeding, we should determine whether these roots are real or complex.

Since T_J is real symmetric, its eigenvalues μ are real.

Furthermore, we claim that

$$-1 < \mu < 1.$$

Indeed, the maximum absolute row-sum of T_J is at most 1, so

$$|\mu| \leq \rho(T_J) \leq 1.$$

Equality cannot occur. To see this, suppose $|\mu| = 1$ and choose an eigenvector component v_i of maximum absolute value. The recurrence

$$2\mu v_i = v_{i-1} + v_{i+1}$$

shows that equality in the triangle inequality forces neighboring components to have the same maximum absolute value. Repeating this argument toward an endpoint eventually contradicts the boundary condition $v_0 = v_{n+1} = 0$.

Thus,

$$\boxed{-1 < \mu < 1.}$$

Consequently,

$$\mu^2 - 1 < 0.$$

The characteristic roots are therefore complex conjugates. We can rewrite them as

$$z_{1,2} = \mu \pm i\sqrt{1-\mu^2},$$

where $i^2 = -1$.

Notice something interesting:

$$\begin{aligned} |z_{1,2}|^2 &= \mu^2 + (1-\mu^2) \\ &= 1. \end{aligned}$$

Hence,

$$\boxed{|z_1| = |z_2| = 1.}$$

Thus, both characteristic roots lie on the unit circle in the complex plane.

Introducing the trigonometric parameter.

Because $-1 < \mu < 1$, there exists a unique angle $\theta \in (0, \pi)$ such that

$$\boxed{\mu = \cos\theta.}$$

This substitution is useful because

$$\begin{aligned} \sqrt{1-\mu^2} &= \sqrt{1-\cos^2\theta} \\ &= \sqrt{\sin^2\theta} \\ &= \sin\theta. \end{aligned}$$

Here we used the fact that $\sin\theta > 0$ for $0 < \theta < \pi$.

Substituting into the characteristic roots gives

$$z_{1,2} = \cos\theta \pm i\sin\theta.$$

Recall Euler's formula,

$$e^{i\theta} = \cos\theta + i\sin\theta,$$

and similarly,

$$e^{-i\theta} = \cos\theta - i\sin\theta.$$

Therefore,

$$\boxed{z_1 = e^{i\theta}, \qquad z_2 = e^{-i\theta}.} \tag{55}$$

We now see why introducing $\mu = \cos\theta$ was natural: the characteristic roots lie on the unit circle, and the angle θ provides a convenient way to represent them.

Constructing the general solution.

Since the characteristic equation has two distinct roots, the general solution of the second-order recurrence is

$$v_k = C_1 z_1^k + C_2 z_2^k.$$

Substituting (55), we obtain

$$v_k = C_1 e^{ik\theta} + C_2 e^{-ik\theta}.$$

Using Euler's formula again,

$$\begin{aligned} v_k = C_1\big(\cos(k\theta) + \mathrm{i}\sin(k\theta)\big) \\ + C_2\big(\cos(k\theta) - \mathrm{i}\sin(k\theta)\big). \end{aligned}$$

Collecting the cosine and sine terms,

$$v_k = (C_1 + C_2)\cos(k\theta) + \mathrm{i}(C_1 - C_2)\sin(k\theta).$$

Writing

$$C = C_1 + C_2, \qquad S = \mathrm{i}(C_1 - C_2),$$

we obtain the equivalent real form

$$\boxed{v_k = C\cos(k\theta) + S\sin(k\theta).} \tag{56}$$

The constants C and S may be chosen real because T_J is a real symmetric matrix.

Applying the boundary conditions.

Recall that our eigenvector must satisfy

$$v_0 = v_{n+1} = 0.$$

Applying the first boundary condition to (56), we obtain

$$\begin{aligned} v_0 &= C\cos(0) + S\sin(0) \\ &= C. \end{aligned}$$

Since $v_0 = 0$, we must have

$$C = 0.$$

Therefore, the eigenvector components reduce to

$$v_k = S\sin(k\theta).$$

Now apply the second boundary condition:

$$v_{n+1} = S\sin((n+1)\theta) = 0.$$

Since the eigenvector is nonzero, we must have $S \neq 0$.

Consequently,

$$\sin((n+1)\theta) = 0.$$

Recall that

$$\sin\alpha = 0 \quad \Longleftrightarrow \quad \alpha = j\pi, \qquad j \in \mathbb{Z}.$$

Therefore,

$$(n+1)\theta = j\pi,$$

which gives

$$\theta = \frac{j\pi}{n+1}.$$

Because $0 < \theta < \pi$, the admissible integers are

$$j = 1, 2, \ldots, n.$$

Thus, the n admissible angles are

$$\boxed{\theta_j = \frac{j\pi}{n+1}, \qquad j = 1, \ldots, n.} \tag{57}$$

Finally, recalling that $\mu = \cos\theta$, we obtain the eigenvalues of the Jacobi iteration matrix:

$$\boxed{\mu_j = \cos\left(\frac{j\pi}{n+1}\right), \qquad j = 1, \ldots, n.} \tag{58}$$

The corresponding eigenvectors have components

$$\boxed{v_k^{(j)} = \sin\left(\frac{kj\pi}{n+1}\right), \qquad k = 1, \ldots, n.} \tag{59}$$

Since the angles θ_j are distinct and cosine is strictly decreasing on $(0, \pi)$, these n eigenvalues are distinct.

Thus, we have determined the complete spectrum of T_J without computing a determinant or performing a numerical eigenvalue calculation.

We can now use these eigenvalues to determine the optimal relaxation parameter for our finite-difference system. Recall that the spectral radius of a matrix is the largest absolute value among its eigenvalues. Since

$$\mu_j = \cos\left(\frac{j\pi}{n+1}\right), \qquad j = 1, \ldots, n,$$

and the cosine function is strictly decreasing on $(0, \pi)$, the eigenvalues of largest absolute value occur at $j = 1$ and $j = n$. Indeed,

$$\mu_1 = \cos\left(\frac{\pi}{n+1}\right),$$

while

$$\begin{aligned} \mu_n &= \cos\left(\frac{n\pi}{n+1}\right) \\ &= \cos\left(\pi - \frac{\pi}{n+1}\right) \\ &= -\cos\left(\frac{\pi}{n+1}\right). \end{aligned}$$

Consequently, the spectral radius of the Jacobi iteration matrix is

$$\boxed{\rho(T_J) = \cos\left(\frac{\pi}{n+1}\right).} \tag{60}$$

We are now ready to apply our theoretical result. Recall that the optimal SOR relaxation parameter is

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho(T_J)^2}}.$$

Substituting the spectral radius obtained above, we have

$$\begin{aligned}\omega_{\text{opt}} &= \frac{2}{1+\sqrt{1-\cos^2\left(\frac{\pi}{n+1}\right)}} \\ &= \frac{2}{1+\sqrt{\sin^2\left(\frac{\pi}{n+1}\right)}} \\ &= \frac{2}{1+\sin\left(\frac{\pi}{n+1}\right)}.\end{aligned}$$

Here, we used the identity $1-\cos^2\theta = \sin^2\theta$ and the fact that $\sin(\pi/(n+1)) > 0$. Thus, for the standard one-dimensional finite-difference matrix with n interior unknowns, we obtain

$$\boxed{\omega_{\text{opt}} = \frac{2}{1+\sin\left(\frac{\pi}{n+1}\right)}.} \tag{61}$$

This is a remarkable result: the optimal relaxation parameter depends only on the number of interior grid points, and no numerical eigenvalue computation is needed. Moreover, as $n \to \infty$, we have $\sin(\pi/(n+1)) \to 0$, and therefore $\omega_{\text{opt}} \to 2$.